DHAR

Bare Metal. Absolute Control.

VOLUME II — ENGINEERING ROADMAP

STAGE-EPSILON

"Deploying Dhar — trustworthy enough for infrastructure that can't fail quietly."

DOCUMENT	Stage-epsilon — Deploying Dhar
VERSION	1.1
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume II
DEPENDS ON	Stage-delta, Amendments A-1 and A-2
FEEDS INTO	Tempering Dhar (Volume III)

1. Mandate

Make Dhar trustworthy enough for production use, including the critical-infrastructure and defence use cases Spec0 §5 names. This is the stage Tempering Dhar (Volume III) leans on most heavily, and the last of the five engineering stages before work moves to the ecosystem and program-management volumes.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1, §5.2; Amendment A-1). For Stage-epsilon that means a release is not a file but a claim, and a claim is allowed only if the gate behind it has passed (Tempering Dhar §11): reproducible, listed, signed, tested for constant time, fuzzed, and cross-compiled to every target the Canon names.

2. Scope

- **Cross-compilation** to **every committed target** (Amendment A-2 §3.1), from one source tree, with a matrix of host and target used for release testing.
- **Debug information** generation for source-level debugging.
- **Profiling hooks** built into the runtime.
- **Static analysis** tooling beyond the compiler's own type checking.
- **Fuzzing infrastructure**, covering both the compiler's own front end and compiled Dhar programs.
- **Reproducible builds**: bit-for-bit identical output from the same source on different machines.
- **SBOM generation** for every release, built directly from Kun's own dependency-resolution records (Stage-delta §3.7) rather than reconstructed separately after the fact.
- **Signing and supply-chain tooling**, with a documented verification path.
- **Release gates** (§3.2): the two-compiler check of Amendment A-1, S4; constant-time timing tests and fuzzing in continuous integration; a migration path for post-quantum algorithms.
- **Hosted services** (§3.1): the compile service behind the in-browser playground (Tooling Dhar §5).

3. Required Architecture

Every back end, including the back end of every committed target (Stage-beta §4.2), **must** consume the same Universal IR Stage-beta and Stage-gamma already established (Spec0 §4). A back end that requires changes to the IR itself, the parser, or the type system is a violation of the independence commitment in Spec0 §2.2 — the whole point of the IR boundary is that a new target is only ever a new back end module and a new target table (Amendment A-1, R4), added by revising Amendment A-2 §3.

3.1 Hosted services

The in-browser playground (Tooling Dhar §5) lets anyone try Dhar in a web page with nothing to install. Its first version runs a compile service; this section states what that **service** must meet. It requires the WebAssembly target at gate G1 (Amendment A-2 §7).

1. **Sandboxed and stateless.** Each request is compiled in its own isolated sandbox that is discarded afterwards. The service **must not** store submitted source code, compiled output or the identity of the requester beyond the request; its logs carry no source code.
2. **No network from the sandbox.** The compiler in the sandbox has no network access and no access to any file other than its own inputs. Compiled code never runs on the service: the module runs in the visitor's browser, in the browser's own WebAssembly sandbox, with only the WASI imports of the committed target (`fd_write`, `fd_read`, `proc_exit`) provided by the page.
3. **Limits.** CPU time, memory, size of source, size of output and request rate per client are all limited. A request that exceeds a limit is stopped with a defined message, not left to fail by timeout.
4. **Abuse handling.** The service can refuse or rate-limit a client, and the terms it operates under are Hallmark of Dhar §8; what it collects is Hallmark §7.
5. **Later version.** When Stage-beta's compiler targets WebAssembly, the compiler itself runs in the browser and the compile service is no longer needed for compilation; this section then applies to whatever hosted service remains.

Each limit has an acceptance test (§4): a program, or a request, that tries to exceed it must be stopped with the defined message.

3.2 Release gates

1. **Two compilers, one source.** For a release of the compiler, the check of Amendment A-1, S4 (Stage-beta §4.1) is a gate: Stage-alpha, frozen and archived, compiles the Stage-beta source; that result compiles it again; the second result **must** equal the Stage-beta binary being released. A release whose binary differs is not released. This is the check known as diverse double-compilation (Tempering Dhar §5.3).
2. **Reproducible, listed, signed.** Two independent builds on two machines are bit-for-bit identical; the release carries a bill of materials built from Kun's records, separating oracle dependencies (Stage-delta §3.8); every release binary is signed.
3. **Constant-time tests in continuous integration.** Every secret-handling routine of `std.crypto` is run on two classes of input at least a million times each and the timing distributions compared with a published statistical method (Amendment A-1 §9, S3); a significant difference fails the build.
4. **Fuzzing in continuous integration**, against the compiler's front end and against compiled programs.
5. **Post-quantum migration.** The `std.crypto` interfaces **must** let a program change algorithm without changing its structure (algorithm agility), so that a deployment can move from a classical algorithm to a post-quantum one (Amendment A-1, Q1) by configuration and a rebuild.

4. Acceptance Criteria

1. A single, unmodified source tree cross-compiles to working binaries on every committed target (Amendment A-2 §3.1), and each passes the conformance battery (Stage-alpha §3.3).
2. Two independent builds of the same source, on two different machines, are bit-for-bit identical.
3. Every tagged release ships with a generated SBOM.
4. Every release binary is signed, with a documented, independently verifiable path.
5. Fuzzing infrastructure runs continuously against the compiler's own front end (Stage-beta/gamma) as well as against compiled Dhar programs.
6. The two-compiler check of §3.2 passes for every release of the compiler.
7. The timing tests of §3.2 run in continuous integration for every secret-handling routine, and a regression fails the build.
8. For each limit of §3.1, a test submits a request that exceeds it and confirms it is stopped with the defined message; a test confirms that no submitted source is retained.
9. A program accepted by the playground compiles unchanged for every other committed target with the same observable behaviour.

5. Handoff

Once Stage-epsilon's acceptance criteria are met, Dhar is specified as production-ready. Ongoing work moves to the ecosystem volumes (Volume III) and program management (Volume IV, The Pact of Developing Dhar) rather than further core-engineering stages — the five-stage roadmap that began at Stage-alpha ends here.

Revision log

Stage-epsilon — Deploying Dhar · v1.1 · October 26, 2026 · Ratified.

VERSION	DATE	CHANGE
v1.0	September 6, 2026	First ratified edition.
v1.1	October 26, 2026	Amendment A-2: cross-compilation for every committed target (§2, §4). Amendment A-1: release gates S4, timing tests, post-quantum migration (§3.2); the hosted compile service of the in-browser playground (§3.1); acceptance criteria (§4).